

x402-attest

Signed Policy Decisions for Agent-to-Agent Payments

rudimentall1 · August 2026 · Proof of Concept

Abstract

The x402 protocol lets an HTTP resource demand payment before serving a request, which means an autonomous agent making x402 payments needs a policy layer deciding whether a given payment offer is acceptable — and a way to prove, later, that a specific decision was really made under a specific policy. x402-attest evaluates a real PAYMENT-REQUIRED offer against operator-defined policy and signs the resulting decision. This paper is intentionally direct about its current scope: it is explicitly labeled a proof of concept, not a production payment component.

1. What x402 Payments Need

An x402 PAYMENT-REQUIRED response describes one or more accepted payment offers — asset, amount, recipient. Before an agent commits to any offer, three questions need answering: is this amount within the operator's policy, is this recipient acceptable, and — if the decision is disputed later — can anyone verify it was genuinely made this way, without re-trusting whichever service made it.

2. Real Header Parsing

x402_hook.py parses an actual base64-encoded PAYMENT-REQUIRED header: the accepts[] array, atomic-unit amounts, the payTo address, and the asset's decimals from its extra.name field where present. Consistent with this author's other projects, an unrecognized asset is refused rather than guessed at:

```
--- Scenario 3: unrecognized asset --- refuses to guess rather than misjudge ---  
verdict: BLOCK (cannot evaluate: no known decimals for asset  
  0xSomeUnknownTokenContract (extra.name='SomeRandomToken') --  
  refusing to guess)
```

Live output from demo_x402.py.

3. Policy Evaluation and Signing

Each parsed offer is evaluated against a YAML policy (maximum amount, recipient allowlist, rate limits) and the resulting decision is signed with Ed25519, independently verifiable, and tamper-evident — altering any field of a signed decision invalidates the signature.

```
--- Scenario 2: large payment (100 USDC), exceeds max_amount in policy.yaml ---  
verdict: BLOCK (amount 100.0 exceeds max_amount 50.0)  
-> a real x402 client would STOP HERE and never construct a PAYMENT-SIGNATURE
```

Live output from demo_x402.py.

4. Explicit Scope Boundary: PAYMENT-SIGNATURE

x402's PAYMENT-SIGNATURE response — the client's EIP-3009 authorization actually moving funds — is not constructed or signed by this project, and that is a deliberate boundary rather than a missing feature to be filled in casually. An EIP-3009 authorization is cryptographically equivalent to a signed transfer of the payer's real funds; producing one requires the payer's private key inside the tool doing the producing.

This project's signing is confined to attesting to its own policy decision — the same category of signing used by this author's OAA specification — and does not extend to constructing value-moving authorizations on a payer's behalf.

5. Honest Status

This project is labeled a proof of concept in its own documentation, and this paper preserves that framing rather than upgrading it for presentation purposes. Explicitly open:

- Rate-limit state is in-memory only and does not survive a restart.
- PAYMENT-SIGNATURE construction (EIP-3009) is not implemented — see Section 4.
- No key rotation or key management story.
- No currency conversion.

6. Conclusion

x402-attest demonstrates the specific, narrow claim that a payment policy decision can be made verifiably and honestly against a real protocol header, without overreaching into the payment-authorization half of the problem that a private key ought to gate. Source: github.com/rudimentall1/x402-attest